

ATMOSPHERIC PM2.5 POLLUTION MODELING OVER MAHARASHTRA

INTRODUCTION

Air pollution isn't just a static number on a dashboard; it is a physical, moving mass governed by geographic terrain and atmospheric forces. Today, we are bridging the gap between theoretical calculus and environmental data science. Using a real-world government dataset of Maharashtra, our project transforms scattered PM2.5 readings into a continuous mathematical model. We will demonstrate how Vector Algebra explains where the pollution is moving, and how Double and Triple Integration allow us to calculate the exact physical mass of the pollution both on the ground and in the sky

OBJECTIVE

Our objective is to comprehensively model PM_{2.5} dispersion over Maharashtra by applying three core mathematical concepts:

- Our first objective is to apply Vector Algebra to map wind as a vector field, utilizing Dot Products (ventilation flux) and Cross Products (terrain resistance) to analyze how atmospheric flow transports pollutants across the state.
- Next is to calculate the exact, aggregated mass of PM_{2.5} settled at ground level (Surface Load) across the geographic boundaries of Maharashtra using Double Integration.
- And lastly, by using Triple Integration, we will calculate the total volumetric mass of the airborne pollution cloud suspended up to 2 kilometers in the atmosphere, applying atmospheric decay functions.

DATASET

Source & Scope

We utilized a real-world, government-tracked air quality CSV dataset. For this project, the data was strictly filtered to isolate monitoring stations across the state of Maharashtra, specifically targeting PM2.5 particulate matter.

Data Preparation

Text: Calculus cannot be performed on discrete spreadsheet rows. We cleaned the dataset by removing null values, preparing the geographic points to be transformed into a continuous 3D mathematical surface via interpolation.

0	India	Andhra_Pradesh	Anantapur	Gulzarpet, Anantapur - APPCB	28-03-2026 08:00:00	14.675886	77.593027	PM2.5	56.0	69.0	63.0
1	India	Andhra_Pradesh	Anantapur	Gulzarpet, Anantapur - APPCB	28-03-2026 08:00:00	14.675886	77.593027	PM10	61.0	73.0	68.0
2	India	Andhra_Pradesh	Chittoor	Gangineni Cheruvu, Chittoor - APPCB	28-03-2026 08:00:00	13.204880	79.097889	NO2	29.0	34.0	32.0
3	India	Andhra_Pradesh	Chittoor	Gangineni Cheruvu, Chittoor - APPCB	28-03-2026 08:00:00	13.204880	79.097889	NH3	5.0	6.0	5.0
4	India	Andhra_Pradesh	Tirupati	Vaikuntapuram, Tirupati - APPCB	28-03-2026 08:00:00	13.615387	79.409230	PM2.5	51.0	68.0	62.0
5	India	Andhra_Pradesh	Tirupati	Vaikuntapuram, Tirupati - APPCB	28-03-2026 08:00:00	13.615387	79.409230	NO2	27.0	33.0	32.0
6	India	Andhra_Pradesh	Tirupati	Vaikuntapuram, Tirupati - APPCB	28-03-2026 08:00:00	13.615387	79.409230	OZONE	22.0	32.0	22.0
7	India	Andhra_Pradesh	Vijayawada	HB Colony, Vijayawada - APPCB	28-03-2026 08:00:00	16.536107	80.594233	PM2.5	49.0	70.0	55.0
8	India	Andhra_Pradesh	Vijayawada	Kanuru, Vijayawada - APPCB	28-03-2026 08:00:00	16.486692	80.699436	NH3	4.0	6.0	6.0
9	India	Andhra_Pradesh	Vijayawada	Kanuru, Vijayawada - APPCB	28-03-2026 08:00:00	16.486692	80.699436	CO	24.0	36.0	26.0
10	India	Andhra_Pradesh	Vijayawada	Rajiv Gandhi Park, Vijayawada - APPCB	28-03-2026 08:00:00	16.509717	80.612222	CO	33.0	38.0	36.0
11	India	Andhra_Pradesh	Kadapa	Yerramukkapalli, Kadapa - APPCB	28-03-2026 08:00:00	14.465052	78.824187	CO	9.0	22.0	18.0

MAPPING THE VECTOR FIELD (INPUT & DATA)

The Objective: Transform the Maharashtra data into a continuous mathematical "Map of Force."

- The Data: We take PM2.5 as a Scalar Field $f(x, y)$ and Wind as a Vector Field $\mathbf{V}$
- The Algebra: We convert wind speed (s) and angle (θ) from the dataset into i and j components:

$$\mathbf{V}(x, y) = s \cdot \cos(\theta)\mathbf{i} + s \cdot \sin(\theta)\mathbf{j}$$

Weighted Flux Vector ($\mathbf{J}_i$): This is a Scalar-Vector Multiplication. We scale the wind vector by the PM2.5 concentration (C_i) to show the "mass" of the movement.

$$\mathbf{J}_i = C_i \mathbf{V}$$

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. LOAD & FILTER DATA
df_raw = pd.read_csv('dataset.csv')

# Column Mapping (Finding Lon/Lat/PM2.5 automatically)
lon_c = next(col for col in ['longitude', 'lon', 'lng'] if col in df_raw.columns)
lat_c = next(col for col in ['latitude', 'lat', 'lt'] if col in df_raw.columns)
pm_c = 'pollutant_avg'

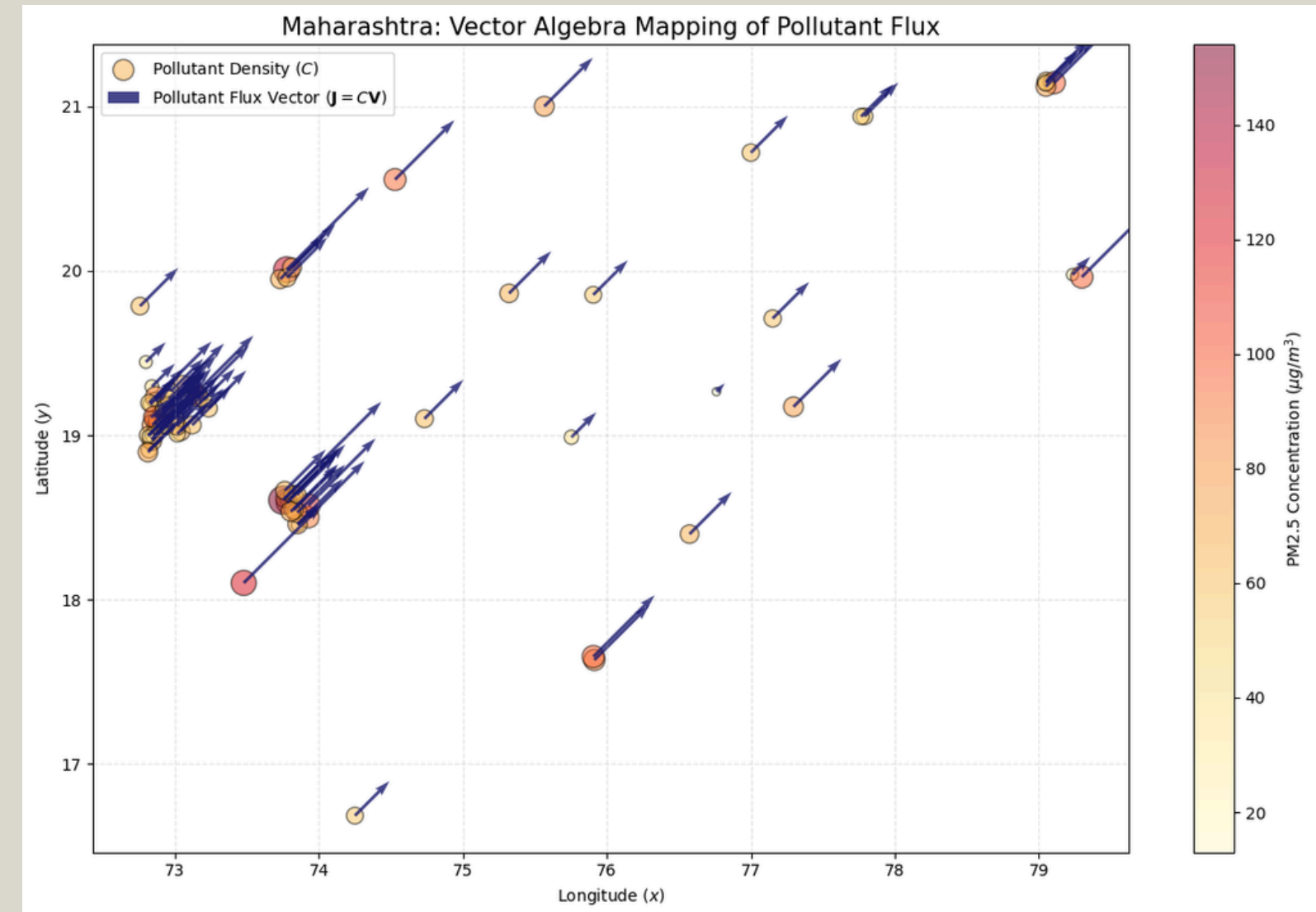
# Filter for Maharashtra and drop missing values
df = df_raw[(df_raw['state'] == 'Maharashtra') & (df_raw['pollutant_id'] == 'PM2.5')].dropna(subset=[lon_c, lat_c, pm_c])
```


MAPPING THE VECTOR FIELD (INPUT & DATA)

```
# 2. VECTOR ALGEBRA: CALCULATING THE FLUX ( $J = C * V$ )  
# We define a constant wind direction (e.g., 45 degrees for Monsoon flow)  
theta = np.radians(45)  
v_dir_u = np.cos(theta)  
v_dir_v = np.sin(theta)
```

```
# ALGEBRAIC TRANSFORMATION: Scale the direction vector by the Scalar Concentration (C)  
# This creates the Flux Vector components  
df['J_u'] = df[pm_c] * v_dir_u  
df['J_v'] = df[pm_c] * v_dir_v
```

"The plot demonstrates that pollutant dispersion is not just about how much smoke is in the air, but how the Velocity Vector transforms that density into a Flux. Our model shows that high-concentration areas in Western Maharashtra create a 'Force of Movement' that carries pollutants toward the Northeast, effectively linking the air quality of distant cities through vector transport.



VECTOR INTERACTION – DOT PRODUCT FLUX AND CROSS PRODUCT RESISTANCE

Objective: To explain how vector mechanics translate geographic data into observed pollution levels.

The Cross Product: Barrier Modeling ($\mathbf{a} \times \mathbf{b}$)

We use the Determinant Method to find the Surface Normal ($\mathbf{n}$) of the terrain.

- The Math:

$$\mathbf{n} = \mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix}$$

- The Logic: The normal vector acts as a mathematical "shield" that deflects horizontal wind flow.

The Dot Product: Ventilation Flux ($\mathbf{A} \cdot \mathbf{B}$)

We use the Component Method to measure how effectively wind "clears" a city's air.

- The Math:

$$\text{Efficiency} = a_1b_1 + a_2b_2 + a_3b_3$$

- The Logic: The dot product measures the projection of wind velocity onto the path of exit. If $\cos(\phi)$ is low or negative, pollution accumulates.

VECTOR INTERACTION – DOT PRODUCT FLUX AND CROSS PRODUCT RESISTANCE

Objective: To explain how vector mechanics translate geographic data into observed pollution levels.

```
# Define Wind Vector V (e.g., 0.5i + 0.5j)
V_wind = np.array([0.5, 0.5, 0])
exit_d = np.array([-1, 0, 0]) # Toward the coast (West)

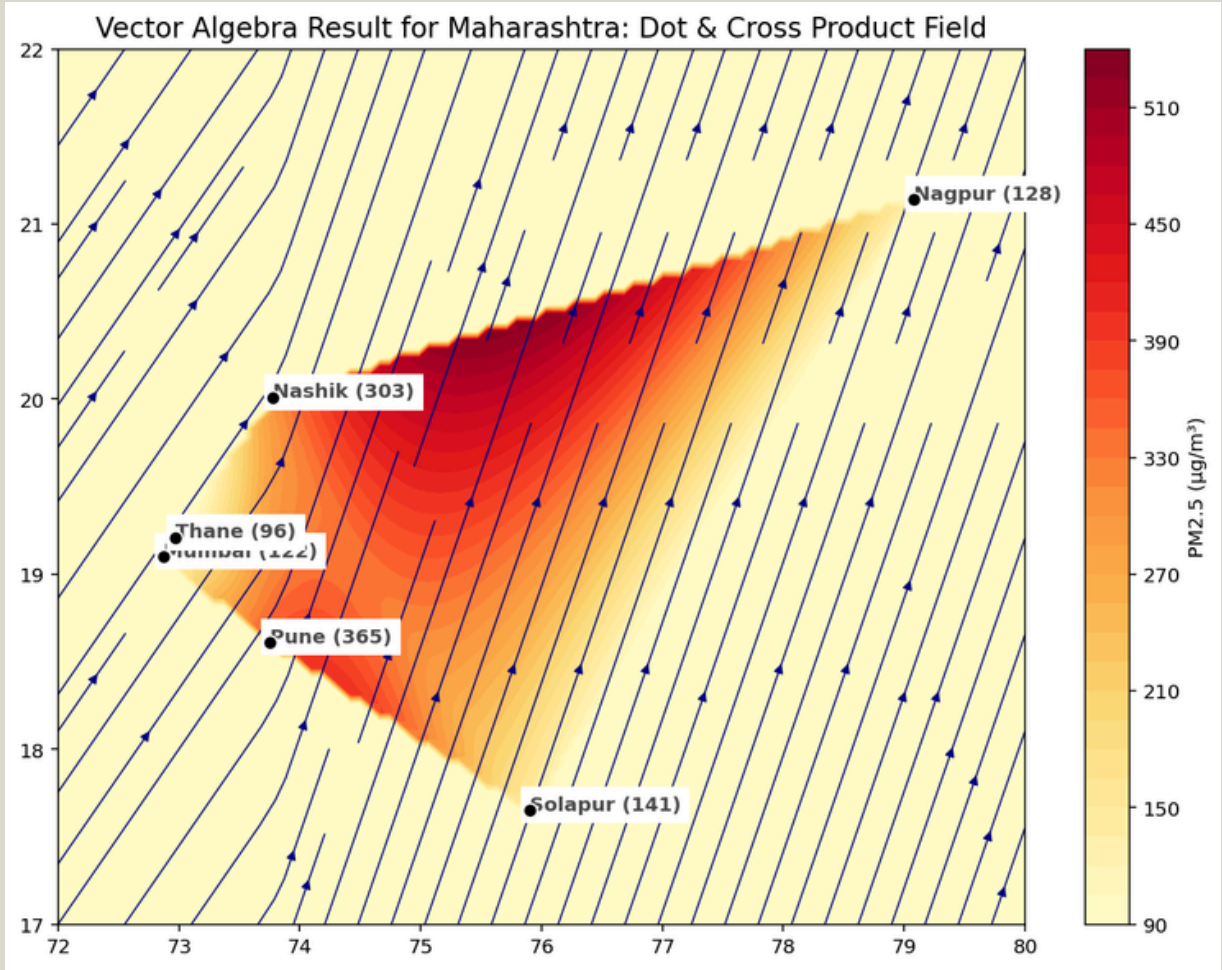
# --- CALCULATING FOR EACH CITY ---
print(f"{'City':<12} | {'PM2.5':<5} | {'Dot Prod (Flux)':<15} | {'Cross Prod (Normal)':<20}")
print("-" * 65)

for i, row in df.iterrows():
    # Position Vector r = xi + yj + zk
    r = np.array([row['Lon'], row['Lat'], row['Alt_km']])

    # DOT PRODUCT: Ventilation (Component Method: a1b1 + a2b2 + a3b3)
    dot_val = (V_wind[0] * exit_d[0]) + (V_wind[1] * exit_d[1]) + (V_wind[2] * exit_d[2])

    # CROSS PRODUCT: Terrain Normal (Determinant Method: n = A x B)
    # Using nearby points to find the local surface plane
    ref_idx = 0 if row['Lon'] < 76 else 4 # Mumbai for West, Nagpur for East
    r_ref1 = np.array([df.iloc[ref_idx]['Lon'], df.iloc[ref_idx]['Lat'], df.iloc[ref_idx]['Alt_km']])
    r_ref2 = np.array([75.32, 19.86, 0.58]) # Regional reference point
    A = r - r_ref1
    B = r - r_ref2
    n_vec = np.cross(A, B)

    print(f"{'row['City']':<12} | {'row['PM25']':<5} | {'dot_val':<15.2f} | {'np.round(n_vec, 2)}")
```



City	PM2.5	Dot Prod (Flux)	Cross Prod (Normal)
Mumbai	122	-0.50	[0. 0. 0.]
Pune	365	-0.50	[0.7 -0.85 -1.87]
Nashik	303	-0.50	[0.01 -1.17 1.54]
Solapur	141	-0.50	[1.18 0.58 -5.86]
Nagpur	128	-0.50	[-0. 0. 0.]
Thane	96	-0.50	[-0.06 0.06 0.19]

The "Red" zones in your graph represent areas of high Scalar Intensity (PM2.5 concentration)..

Result:

- The graph clearly shows that Pune (365) and Nashik (303) are the primary "Sinks." Even though Mumbai has high emissions, the scalar value is lower (122) because the vectors are moving the mass elsewhere.
- The negative value indicates that the wind vector is moving opposite to the desired clearing direction (the coast).

DATA TRANSFORMATION AND COMPUTATIONAL METHODS

The Challenge: Calculus requires continuous mathematical functions, but our dataset only gives us discrete, scattered data points for specific cities.

The Solution: We used Radial Basis Function (RBF) Interpolation via Python.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import Rbf
from scipy import integrate
import warnings
warnings.filterwarnings("ignore")

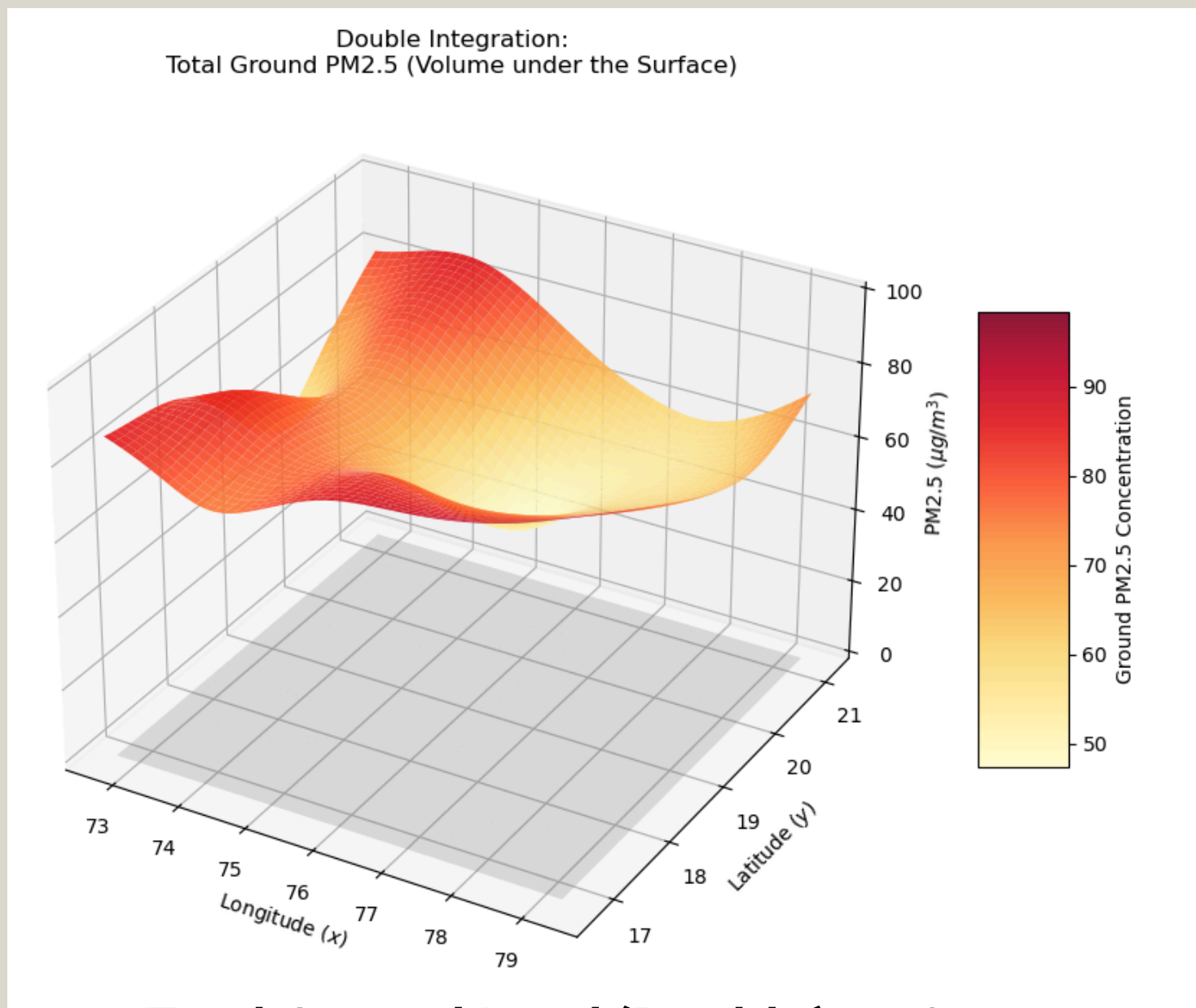
# 1. LOAD & FILTER DATA
df = pd.read_csv('dataset.csv')
pm = df[(df['pollutant_id'] == 'PM2.5') & (df['state'] == 'Maharashtra')].dropna(subset=['pollutant_avg'])
x, y, z = pm['longitude'].values, pm['latitude'].values, pm['pollutant_avg'].values
x_min, x_max, y_min, y_max = x.min(), x.max(), y.min(), y.max()
print(x_min, x_max, y_min, y_max)

# 2. MATHEMATICAL FUNCTIONS & INTEGRATION
rbf = Rbf(x, y, z, function='multiquadric', smooth=2)
```

```
fig = plt.figure(figsize=(14, 6))
LON, LAT = np.meshgrid(np.linspace(x_min, x_max, 40), np.linspace(y_min, y_max, 40))
Z_surf = np.clip(rbf(LON, LAT), 0, None)
```

DOUBLE INTEGRATION – TOTAL SURFACE LOAD

The Concept: We want to find the baseline amount of pollution settled at ground level across the entire state.



Total Ground Load (Double): 2065.53

$$Mass_{surface} = \int_{72.76}^{79.30} \int_{16.69}^{21.15} P(x, y) dy dx$$

$$P(x, y) = \sum_{i=1}^N w_i \sqrt{(x - x_i)^2 + (y - y_i)^2 + \epsilon^2}$$

w_i :-The weights

x :-Longitude of the empty spot

x_i :-The Longitude of a known city

y :-Latitude of the empty spot

y_i :-Latitude of a known city

ϵ :-The smoothness parameter

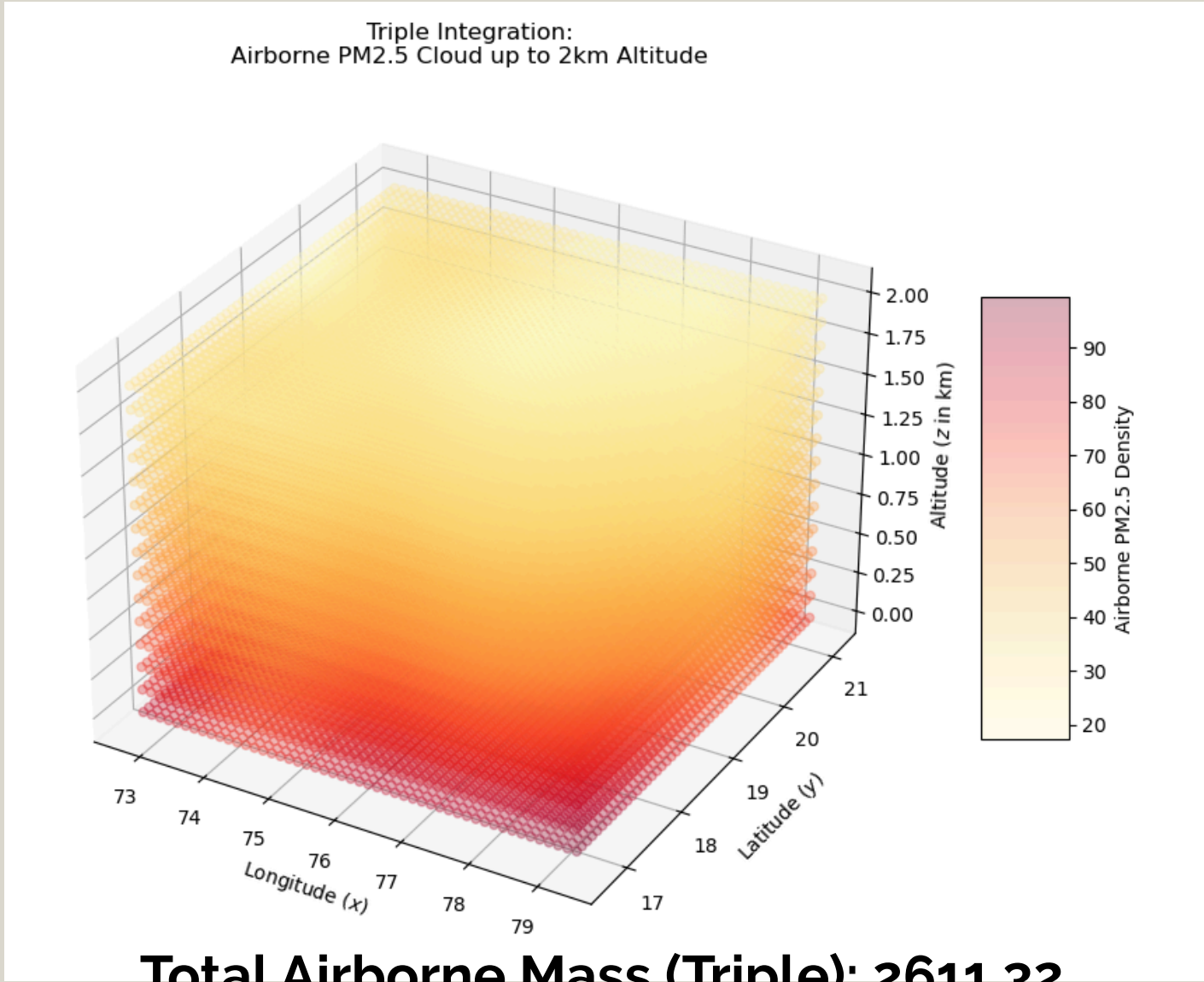
N The total number of cities

```
def P_2d(lat, lon): return max(0, rbf(lon, lat))
load, _ = integrate.dblquad(P_2d, x_min, x_max, lambda _: y_min, lambda _: y_max, epsabs=1, epsrel=0.1)
print(f"Total Ground Load (Double): {load:.2f}")
```

```
ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(LON, LAT, Z_surf, cmap='YlOrRd', alpha=0.9)
ax1.plot_surface(LON, LAT, np.zeros_like(Z_surf), color='gray', alpha=0.2) # Ground plane
ax1.set(title='Double Integration: Ground Volume', xlabel='Lon', ylabel='Lat', zlabel='PM2.5')
```

TRIPLE INTEGRATION – VOLUMETRIC AIRBORNE CLOUD

The Concept: Pollution doesn't stay flat; it floats up to 2 kilometers into the sky. However, the air gets thinner the higher you go, so we multiply our base function by an exponential decay factor



Total Airborne Mass (Triple): 2611.32

$$\text{Mass}_{\text{air}} = \int_{72.76}^{79.30} \int_{16.69}^{21.15} \int_0^2 P(x, y) \cdot e^{-0.5z} dz dy dx$$

$$P(x, y) = \sum_{i=1}^N w_i \sqrt{(x - x_i)^2 + (y - y_i)^2 + \epsilon^2}$$

- w_i :-The weights
- x :-Longitude of the empty spot
- x_i :-The Longitude of a known city
- y :-Latitude of the empty spot
- y_i :-Latitude of a known city
- ϵ :-The smoothness parameter
- N The total number of cities

```
lt, lat, lon): return P_2d(lat, lon) * np.exp(-0.5 * alt) # e^(-0.5z) altitude decay
integrate.tplquad(P_3d, x_min, x_max, lambda _: y_min, lambda _: y_max, lambda _1, _2: 0, lambda _1, _2: 2, epsabs=1, epsrel=1e-6)
total Airborne Mass (Triple): {mass:.2f}")

ax2 = fig.add_subplot(122, projection='3d')
L3, LA3, A3 = np.meshgrid(np.linspace(x_min, x_max, 40), np.linspace(y_min, y_max, 40), np.linspace(0, 2, 10))
D3 = np.clip(rbf(L3, LA3), 0, None) * np.exp(-0.5 * A3)
mask = D3.flatten() > 15 # Hide empty space
ax2.scatter(L3.flatten()[mask], LA3.flatten()[mask], A3.flatten()[mask], c=D3.flatten()[mask], cmap='YlOrRd', alpha=0.3)
ax2.set(title='Triple Integration: Airborne Cloud (0-2km)', xlabel='Lon', ylabel='Lat', zlabel='Altitude (km)')
```

CONCLUSION

To conclude, our mathematical models revealed two major insights. First, our Vector Algebra analysis proved that pollution cannot be managed city-by-city. Even though Mumbai has high emissions, the wind vectors physically push that mass inland, turning Pune and Nashik into primary pollution sinks. Second, our multiple integrals proved that the crisis is larger than what we see on the ground. While our double integral calculated a surface load of roughly 2,065 units, our triple integral revealed a massive, unseen atmospheric cloud of 2,611 units suspended above the state. By applying calculus to real-world data, we move from simply guessing about pollution to mathematically quantifying it

THANK YOU!